TESTING

El testing es un proceso destructivo que tiene como objetivo encontrar defectos, asume que una porción de software puede y tiene defectos y su objetivo es encontrarlos, para que sean resueltos y CONTRIBUIR (no asegurar) a la calidad del producto de software, NO la inyecta al final
Se debe ir con actitud negativa para demostrar que algo que construyo otra persona esta mal
Y el testing es EXITOSO siempre y cuando se hayan encontrado defectos

¿Cuánto testing es suficiente?

Depende de ciertos aspectos, para un producto de software complejo, para probar exhaustivamente toda la combinación de posibilidades conllevaría un tiempo mayor a nuestra vida, por lo tanto, no se puede asegurar la prueba de todas las posibilidades, frente a esto surgen los siguientes criterios:

GOOD ENOUGH: A medida que encontramos defectos y los resolvemos, la tendencia de encontrar defectos va a disminuir, y cuando esta tendencia de encontrar defectos alcance un cierto umbral, cortaremos
O con respecto a un presupuesto enfocado en encontrar pruebas

Proceso del Testing ETAPAS DEL TESTING
Planificación y control: Definiendo estrategia, alcance, riesgos, recursos, etc se obtiene el plan de pruebas que sirve para controlar la toma de decisiones, revisar el resultado del testing y evaluar su cobertura

Analisis y diseño: Se identifican las condiciones de prueba, requerimientos, se diseñan y preparan casos de prueba

Ejecución: Se ejecutan los casos de prueba y se comparan los resultados esperados con los obtenidos

Evaluación y reportes: Se verifican los entregables, se corrigen defectos, y se establecen métricas

Fin de pruebas: Se deja de probar una vez se alcancen las consideraciones de defectos aceptados, generando un informe final.

En la etapa análisis y diseño, se diseñan casos de pruebas que son el artefacto por excelencia del testing, sirven como guía para la ejecución, gracias a esta ejecución obtenemos la evaluación y reporte que sirve para la retroalimentación

Y los artefactos que se obtienen como salida del proceso de testing
-Plan de pruebas
-Casos de prueba
-Reportes de incidentes o defectos
-Informe final

ERROR ≠ DEFECTO ≠ FALLA

Error: Queda en el contexto de la misma fase que ocurrió
Defecto: Pasa de fase, un error de implementación que pasa a la prueba se convierte en defecto
(los defectos pueden tener edad CREO) (El testing busca encontrar defectos y no errores)
Falla: Es un mal funcionamiento del sistema

Los defectos y errores provocan fallas
Puede haber defectos que no provoquen fallas (mal funcionamiento)

No todos los defectos son iguales

Severidad

Consiste en que tanto daño le genera un defecto al software
1- Bloqueante / invalidante = no permite usar el sistema
2- Critico = No puedo cumplir el obj por esta funcionalidad, pero el sistema anda, NO hay plan B
3- Mayor = no anda la funcionalidad, pero hay plan B
4- Menor = hay q resolver, pero no impide cumplir la funcionalidad
5- Cosmético = ortográfica, menús, cosas visuales

Prioridad

Tiene que ver con la urgencia del negocio, la prioridad la asigna el usuario final cuando lo detecta (aunque no es bueno que un cliente detecte defectos)
1- Urgente, no se puede esperar nada 2- Alta 3- Media 4- Baja

Niveles de prueba

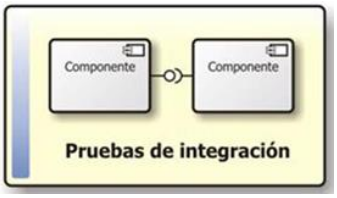
(Dijo que estudiemos los títulos)

Los niveles de prueba determinan la granularidad de la prueba que se está por ejecutar.
En general, primero se comienzan probando componentes pequeños, y luego se integran en pruebas de mayor granularidad. Esto es al revés de cómo se desarrollan los componentes.

primer nivel = testing unitarias → Es llevado a cabo por el desarrollador
Tmb se llama primer nivel de testing (es la excepción de la regla, el desarrollador le hace testing a su propio codigo para saber que esta por entregar con el objetivo de buscar **ERRORES**) se realizan sobre un componente, para verificar que cumpla correctamente con la funcionalidad **Se aplica en el workflow de implementación**



Segundo nivel = testing de integración → Es llevado a cabo por el tester
Se busca encontrar defectos, se prueban las interfaces de componentes
Se busca comprobar la integración y funcionamiento de dos o más componentes que se relacionen entre ellos. Se prueba la comunicación
Se puede hacer con estrategia bottom up, top down o sándwich
El resultado es un BUILD Se hace en el ámbito de la implementación, aunque si se trabaja con integración continua se puede hacer en el workflow de prueba

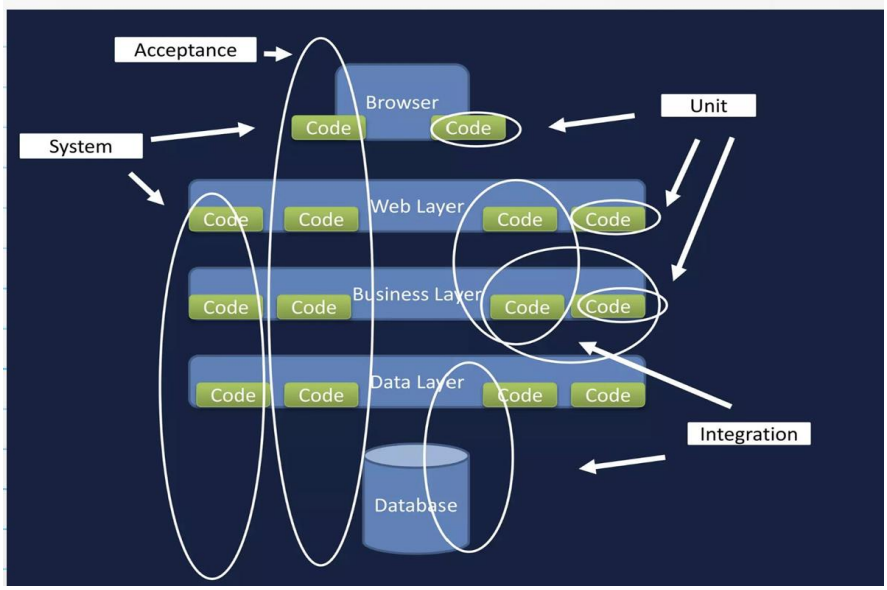


Tercer nivel = testing de sistema/versión → Es llevado a cabo por el tester
Se deben realizar por alguien independiente de los que lo crearon.
Se busca validar el sistema completo, integrado en un entorno que EMULA la producción (no es producción, pero tiene las mismas características)
Testing y desarrollo no deberían tener el mismo jefe Pruebas manuales end to end.
Se llevan a cabo pruebas del funcionamiento real y cotidiano del producto. Abarca requerimientos funcionales y no funcionales.



Cuarto nivel = Pruebas de aceptación → Es llevado a cabo por el PO/cliente/usuario
Se realizan en el workflow despliegue
Debe realizarla el usuario para verificar que se cumple con sus requerimientos manifiestos y expectativas no manifiestas. tiene que estar el equipo presente
Busca que el cliente se familiarice con el producto y su objetivo principal no es encontrar fallas





Ambientes para construcción del SW



Los ambientes de:
- **desarrollo** (hay HW y SW necesario para desa y desple el product)

- **pruebas** (solo tienen acceso los tester y no los dev)

- **preproducción** (ambiente q emula las características del ambiente productivo para comprobar que el producto funcionara una vez desplegado efectivamente)

- **producción**

permiten aislar actividades y gestionar las versiones.
No es obligatorio disponer de un ambiente exclusivo para cada nivel de prueba.
En particular, el ambiente de pruebas de sistema debe asemejarse al de producción tanto como sea posible.
Como minimo son 3 ambientes (Desarrollo, Testing y Producc)

VALIDACIÓN O VERIFICACIÓN

La validación se pregunta: ¿Estamos construyendo el sistema correcto? Controla si el producto SATISFACE las necesidades y expectativas reales del cliente y su negocio.

La verificación consiste en preguntarse: ¿Estamos construyendo el sistema correctamente?
Controla la correspondencia del producto contra las especificaciones técnicas.

Cabe aclarar que NECESIDADES son aquellas cosas que quedan EXPLICITADAS por el cliente, las expectativas son aquello que se ASUME.

ESTRATEGIAS

Al existir infinitas combinaciones de valores es lógico que no se pueda probar todos los casos de una funcionalidad, y por eso surgen diferentes criterios y estrategias para el diseño de casos de prueba que tienen como objetivo maximizar la cantidad de defectos encontrados minimizando el esfuerzo requerido para hacerlo

Como se comento antes, los **casos de prueba** son el artefacto por excelencia del testing y esta compuesto por un conjunto de condiciones y pasos a ejecutar, para garantizar que el resultado esperado sea igual o no al resultado realmente obtenido y de esta manera encontrar defectos

Estrategia Caja Blanca (ES DINAMICO)
Vemos el detalle de implementación de la funcionalidad (codigo) y podemos diseñar el CP, garantizando la cobertura (cubrir todos los IFs, los ELSEs, ramas, etc gracias a que podemos ver el codigo podemos guiar la funcionalidad para donde nos interese)

Estrategia Caja Negra (ES ESTATICO)
No conozco la estructura interna de la implementación, solo la analizamos en termino de entradas y salidas básicamente se identifican las entradas, se define un cierto grupo de valores, se prueba y se comparan las salidas obtenidas contra las esperadas

Lo mejor para el testing es hacer una combinación de estrategias ya que cada una tiene sus pros y contras

METODOS

Los métodos son implementación de una estrategia de forma particular

Caja Negra:

- Basados en especificaciones:

*** Partición de equivalencias**

consiste en analizar cuáles son las diferentes condiciones externas involucradas en el desarrollo de una funcionalidad, estas condiciones externas son entradas y salidas, es decir nos paramos en la funcionalidad y determinamos cuales son todas las entras y salidas posibles
Entradas: cualquier tipo de variable, campo de texto, combo de selección, coordenadas, medición de un sensor, cualquier condición que guía la ejecución de la función
Salidas: Abstraerse de la interfaz → puede ser tanto una luz de un control remoto como un mensaje en pantalla

Luego, de estas condiciones externas analizamos cuales son los subconjuntos de valores posibles que pueden tomar dichas condiciones externas que producen un resultado equivalente

Partición de equivalencia = Clase de equivalencia → Subconjunto de valores que puede tomar una condición externa para el cual si yo tomo cualquier miembro de ese subconjunto el resultado en la ejecución de la funcionalidad es equivalente (no necesariamente igual, equivalente)

*** Analisis de valores límites**

Es una variante de la partición de equivalencias, en vez de seleccionar cualquier elemento como representativo de una CE se seleccionan los valores de los bordes ya que se considera que son los más representativos / los que son más propensos a fallar
CE valida: ingresar con 18/100 años CE invalida: ingresa con 17 años

Basados en la experiencia:

* **Adivinanza de defectos**: Se utiliza conocimiento previo sobre fallas frecuentes para proponer pruebas

* **Testing Exploratorio**: combina aprendizaje, diseño y ejecución de pruebas durante la exploración del sist

Caja Blanca:

Garantizan la cobertura

● **Cobertura de caminos básicos**: Busca garantizar que todos los caminos independientes de la funcionalidad los recorramos al menos 1 vez (pasar por todos los caminos)

Para poder llevar a cabo la prueba:

- Se tiene que poder representar la ejecución mediante un grafo de flujo
- Se calcula complejidad Ciclomática **M = E – N + 2P** **M = cant de regiones + 1**

E = cant aristas N = cant nodos P = cant nodos salida

Complejidad Ciclomática	Evaluación del Riesgo
1-10	Programa Simple, sin mucho riesgo
11-20	Más complejo, riesgo moderado
21-50	Complejo, Programa de alto riesgo
50	Programa no testeable, Muy alto riesgo

● **Cobertura de sentencias**: busca que cada sentencia/instrucción ejecutable del codigo se ejecute al menos una vez durante las pruebas.

Una Sentencia es cualquier instrucción (asignación de variable, invocación de métodos, mostrar msj)

Están las estructuras de control por ejemplo IF/Else no son sentencias

● **Cobertura de decisiones**: busca que cada decisión lógica (estructura de control, ej: un if) produzca tanto el resultado VERDADERO como el FALSO durante las pruebas (independiente mente de si tengo o no sentencia del lado del false).

A cobertura de decisiones no le importa cómo se evalúan las condiciones solo el camino q toma

● **Cobertura de condiciones**: busca que cada condición simple que integra una condición lógica tome los valores VERDADERO y FALSO, independientemente del resultado de la decisión completa.

a > 0 c == 0 ! =

● **Cobertura de decisión/condición**: combina los criterios anteriores, buscando ambos resultados para las decisiones y para cada condición individual. Cobertura múltiple: busca probar todas las combinaciones posibles de valores VERDADERO y FALSO de las condiciones de una decisión.

● **Cobertura múltiple**: ¿combinatoria de todas las posibilidades?

● **Etc**. (hay muchas más, pero le damos bola a estar para el practico)

CICLO DE TEST

Los ciclos de pruebas abarcan la ejecución de la totalidad de los casos de prueba establecidos a una versión del sistema a probar.

Como resultado del ciclo, se obtiene un **reporte de prueba**.

Los incrementos se cuentan desde el 0 en adelante, y en el mejor de los casos habrá 2 ciclos, el 0 y el 1 para correcciones (por el supuesto de que SI hay defectos).

REGRESIÓN

Cuando se finaliza un ciclo de pruebas, esta la posibilidad de que se haya corregido algún error entonces hay que realizar nuevamente una verificación de todos los casos de prueba para que pasen sin problema

La automatización de las pruebas de regresión, por ejemplo, mediante CI/CD, facilita volver a ejecutarlas sistemáticamente

PRINCIPIOS DEL TESTING

● El testing muestra presencia de defectos, pero no puede demostrar ausencia de estos (que no se encuentren no garantiza que no existan).

● Es IMPOSIBLE que sea exhaustivo, es inviable económicamente y por tiempo, para ello se aplican criterios de aceptación para definir el alcance adecuado.

● El testing temprano detecta defectos en etapas tempranas, reduciendo exponencialmente los costos de corrección.

● Los defectos tienden a agruparse en un número reducido de módulos, aplicando el principio de Pareto 80/20 → el 80% de los defectos se encuentra en el 20% de funcionalidades

● Paradoja del pesticida: Si los mismos casos de prueba se ejecutan repetidamente, eventualmente dejarán de encontrar nuevos defectos. Para mantener la efectividad, los casos de prueba deben revisarse y actualizarse periódicamente.

● El testing es DEPENDIENTE del contexto, las estrategias y técnicas a utilizan dependerán del tipo de SW, no es lo mismo testear un juego que un sistema bancario

● Falacia de la ausencia de errores → el testing asume que siempre hay defectos

● Un programador no debería probar su propio codigo; una unidad de programación no debería probar sus propios desarrollos; el esfuerzo del testing debe planificarse

● El costo del testing está entre un 30% y 50% (retrabajo) del valor del producto

Smoke test

Conjunto breve de pruebas iniciales sobre una build para comprobar que sus funcionalidades principales están operativas y que no presenta fallas catastróficas antes de ejecutar pruebas más profundas. Puede repetirse cada vez que se recibe una nueva build. Su nombre proviene de la electrónica.

Tipos de pruebas

Cabe aclarar que lo funcional y no funcional son los requerimientos que se prueban.

Testing Funcional (para probar Req funcionales): Evalúa qué hace el sistema, analizando funciones y reglas de negocio frente a las especificaciones. Son basados en requerimientos y en procesos de negocio.

- **Prueba de Operación Normal** (testing funcional): Buscar las pruebas que pasan de las US, intención de usar el sistema en los términos que hay que usarlo. Son el camino feliz.
- **Pruebas negativas**: Relacionada a aquellas pruebas en donde el sistema debería fallar, al uso indebido del software.
- **Pruebas de proceso de negocio** (Circuito): Se concatenan las funcionalidades de las US para validar el proceso completo. (Puede ser un solo caso de prueba o un conjunto de casos de prueba para cada US y la suma genera la prueba de PN).
- **Documentacion**: Que los manuales de usuario, de configuración, sean consistentes con el funcionamiento del sistema

Los casos de prueba pueden servir para distintas pruebas, pero no en el mismo instante

Testing No funcional: Evalúa cómo funciona el sistema, son dependientes del entorno donde el sistema está instalado (rendimiento, seguridad, usabilidad, fiabilidad, etc.).

Prueba de Performance → Se mide el tiempo de respuesta desde que el usuario hace una petición hasta que el sistema responde, no se calcula solo el tiempo si no tmb los recursos utilizados
Prueba de carga → Se observa a lo que se comprometió el equipo como configuración de hardware y se comprueba si funciona (cantidad de usuarios, puestos activos al mismo tiempo, etc)
Prueba de Estrés → Se fuerza el sistema para ver hasta donde soporta y ver cómo queda luego de explotar
~~**Prueba de Carga** → Concurrencia de usuario y uso de BD~~
Prueba de Usabilidad → Relacionadas con la experiencia de usuario ~ si el usuario le va a gustar el producto
Accesibilidad →Usabilidad para personas con capacidades diferentes (ceguera)
Mantenimiento → Frente a un cambio del producto que tanto esfuerzo nos conlleva volver a testear la aplicación, básicamente evaluar el esfuerzo que conlleva realizar un cambio
Fiabilidad → Que tan confiable es el sistema (que tanto se pueden depender de sus resultados)
Portabilidad → Controlar que tanto soporta el producto moverse de entornos (distintos SO, plataforma, navegadores, etc)
Seguridad → Lógica y física; ejemplos serían los permisos, datos a los que se puede acceder (física seria lo que puede hacer en persona; la lógica son lo que se hace sobre el sistema)
Lógica → Protege lo que se puede hacer dentro del sistema
Física → Protege a los recursos tecnológicos, equipos, instalaciones
Configuración → verifican que el software funcione correctamente bajo distintas configuraciones de hardware, software y entornos de ejecución. Busca detectar problemas de compatibilidad.
Interfaces de usuario → Son más técnicas, relacionado a los componentes con los que el usuario puede interactuar, controlando que dichos componentes respondan correctamente

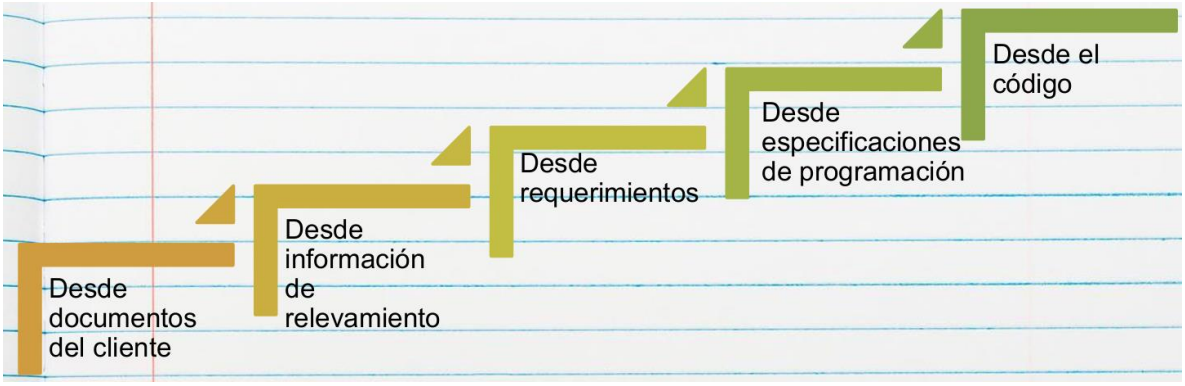
En los casos de prueba se pueden mezclar pruebas de requerimientos funcionales y no funcionales, pero no es óptimo

Casos de prueba

Unidad básica de la actividad de testing, siendo el artefacto principal que se desarrolla durante la fase de Identificación y Especificación. Es el conjunto formal de condiciones previas, datos de entrada, secuencia de pasos, y resultados esperados cuya finalidad es la de descubrir la mayor cantidad de defectos posibles, con el minimo esfuerzo y tiempo posible.
Los casos de prueba deben ser REPRODUCIBLES y REUTILIZABLES siempre y cuando los requerimientos no cambien (es decir, deben dar el mismo resultado bajo las mismas condiciones).

Los casos de prueba se diseñan durante el ANÁLISIS Y DISEÑO de testing y sirven como guía para su EJECUCIÓN. Derivan de documentos del cliente, información de relevamiento, requerimientos, especificaciones y código. Su diseño sistemático permite probar más condiciones relevantes con menos esfuerzo.

Niveles a partir de los que se pueden crear casos de prueba



Estructura de un caso de prueba

ID: Identificador único, generalmente numérico.

Prioridad: Orden de ejecución según criticidad y capacidad de detectar defectos.
Alta: Camino feliz o funcionamiento principal
Media: combinaciones relevantes
Baja: validaciones secundarias

Nombre: Título descriptivo de lo que se probará.

Objetivo: Funcionalidad, regla o condición que se busca comprobar.

Precondiciones: Estado inicial y características del ambiente, expresados mediante datos CONCRETOS para poder reproducir el caso.

Pasos: Operaciones ordenadas y numeradas, indicando quién actúa y qué hace (por ejemplo: «El usuario ingresa...»).

Datos de entrada: Valores específicos utilizados durante la ejecución.

Resultado esperado: Salida o comportamiento que debe producir el sistema, de acuerdo con los requerimientos.

TDD: Test Driven Development

Consiste en el desarrollo del SW guiado por pruebas automatizadas.
Primero se especifica un comportamiento mediante un test que falla; luego se escribe el código mínimo para pasarlo y finalmente se mejora el diseño sin alterar el comportamiento.
El test guía la implementación y proporciona feedback temprano.

El Objetivo es avanzar mediante ciclos cortos, mantener el código comprobable y favorecer un diseño simple; una alta cobertura puede ser consecuencia, pero no sustituye el diseño de buenas pruebas.

LEYES DE TDD

1) No escribir código de producción antes de tener una prueba automatizada que falle
2) No escribir más código de prueba que el suficiente para producir un fallo.
Un error de compilación también puede contar como fallo
3) No escribir más código de producción que el necesario para hacer pasar la prueba

Ciclo de TTD → RED – GREEN – REFACTOR

RED: Escribir una prueba automatizada para expresar un comportamiento esperado y verificar que falla por el motivo correcto. Un CdP puede estructurarse con precondiciones, pasos y resultado esperado (assert).

GREEN: Escribir el **mínimo** código de producción necesario para que pase la prueba, sin anticipar funcionalidades no requeridas.

REFACTOR: Mejorar la estructura, legibilidad y diseño del código, eliminando duplicaciones **sin modificar** su comportamiento observable; ejecutar las pruebas para comprobarlo.

Se repite el ciclo agregando casos progresivamente más exigentes. Mediante nuevos ejemplos y triangulación se pasa de soluciones específicas a implementaciones generales.

Patrones y buenas prácticas para hacer que el ciclo siga fluyendo y evite estancarse en la etapa de desarrollo.

Patrones red: Ayudan a definir qué se quiere hacer.

- **Starter test**: Punto de partida, elegir la prueba más trivial, facil y obvia.
- **One Step Test**: Seguir con el siguiente test en la línea de complejidad,
- **Explanation Test**: Para etapa madura, permite debatir sobre el comportamiento esperado del sistema representado en un test.
- **Learning Test**: Representar procesos de aprendizaje a través de pruebas, entender funcionamiento del sistema para poder hacer integraciones.
- **Another Test**: Permite anotar prox tests como pendientes, pero sin perder el foco de la tarea actual
- **Regresion Test**: Definir un test que replique los pasos de ejecución de un problema para que no vuelva a ocurrir en el futuro. Replicar lo que sucedió para evitar que vuelva a pasar.

Patrones Green: Ayudan a hacer pasar las pruebas con la implementación mínima necesaria.

- **Fake it 'til you make it:** Hardcodear la respuesta en la implementación para obtener el resultado esperado y hacer que la prueba pase rápidamente.
- **Triangulación:** Agregar ejemplos o casos de prueba distintos que obliguen a generalizar una solución demasiado específica
  (por ejemplo, tras hacer pasar suma (2, 3) = 5 con un valor fijo, agregar suma (4, 6) = 10 exige implementar la suma real).
- **Obvious Implementation:** Escribir la implementación directa y correcta desde el primer momento cuando la solución es evidente y sencilla.
- **One to Many:** Evolucionar la solución desde el manejo de un solo elemento hacia colecciones mediante iteraciones graduales

Prácticas de diseño:

Test list: Antes de desarrollar, hacer listado de pruebas
Isolated test: busca que los patrones sean los más independientes del resto, pruebas unitarias
Test first: Primero escribir prueba y luego codigo
Assert Test: Arrancar del resultado (primero resultados, luego pasos, luego precondiciones
Test Data: La información que se debe usar debe ser lo más sencilla y facil de entender
Evident Data: Relación entre entradas y salidas sea lo más obvia posible

Testing Agile

El testing agile es un enfoque en el que las actividades de prueba se **integran continuamente** en el proceso de desarrollo, en lugar de concentrase en la fase final como en el tradicional.
Busca obtener feedback temprano para detectar defectos rápidamente y contribuir a la calidad durante todo el desarrollo.

Pilares

● Automatizar todo lo que se puede automatizar, entendiendo que el tiempo será un problema y restricción → Permite la regresión (ya que si esta automatizado es mucho más facil verificar)
● Se busca cambiar la idea del testing como destructivo
● Se busca trabajar para la prevención
● Se busca entender las reglas de negocio (lo que se está probando, apuntar a validar por sobre verificar). → Que genere valor
● todo el equipo es el responsable de la calidad, esto evita que toda la culpa recaiga en el equipo de testing



PRINCIPIOS

1. **Testing se mueve hacia adelante en el proyecto**: Significa que se comienza a ejecutar lo antes posible para detectar rápidamente los problemas.
2. **Testing no es una fase**: Se lleva a cabo de forma íntegra en el proceso de desarrollo, de forma continua
3. **Todos hacen testing**: La responsabilidad de ofrecer calidad no recae solo en el equipo de testing si no que todos los integrantes deben detectar fallos
4. **Reducir la latencia del feedback**: Reducir tiempo de retroalimentación
5. **Las pruebas representan expectativas**: Es cómo se espera que funcione el sistema ante determinadas condiciones
6. **Mantener el codigo limpio y corregir los defectos rápido**: Evitar acumulación de defectos y reducir retrabajo
7. **Reducir la sobrecarga de documentacion de las pruebas**: Evitar documentacion excesiva
8. **Las pruebas son parte del Done**: Considerar como terminada a una US si cumple los criterios de calidad y las pruebas establecidas
9. **De probar al final ha conducido por pruebas**: En lugar de construir primero y probar después, las pruebas pueden utilizarse para guiar al desarrollo desde el inicio

PRACTICAS CONCRETAS

Pruebas unitarias y de integración automatizadas: Verificar componentes y sus interacciones frecuentemente
TDD: Desarrollar codigo guiado por pruebas
Regresión automatizada a nivel de sistema: Detectar defectos introducidos por los cambios
ATDD (Desarrollo conducido por pruebas de aceptación): Desarrollar a partir de las pruebas de aceptación.
Testing exploratorio: Explorar el producto e identificar comportamientos inesperados
Versionado de pruebas junto al codigo: Mantener la correspondencia entre pruebas y versiones

TDD VS ATDD

TDD busca escribir primero las pruebas y luego el codigo minimo para superarlas para finalmente refactorizar (Red - Green - Refactor) → Concentrado en lo TECNICO
ATDD: Primero se definen las pruebas de aceptación y luego se establece qué comportamiento debe satisfacer el sistema según los Req del negocio. → Se enfoca en las necesidades y expectativas del cliente, en lo que lo satisface.

ROL DE LOS TESTER

En los enfoques tradicionales, el tester recibe el producto una vez ha sido desarrollado; en los ambientes agiles, el tester se integra al equipo y participa durante todo el desarrollo, colaborando en la rpevencion y detección temprana de defectos. Entre sus responsabilidades recae:
- Participar en la Planning y comprensión de requerimientos
- Colaborar con desarrolladores y representantes del negocio
- Ayudar a definir criterios de aceptación
- Diseñar y ejecutar pruebas, manuales y automatizadas
- Validar el Done de las historias
- Contribuir a la mejora de los procesos de testing

Cuadrantes

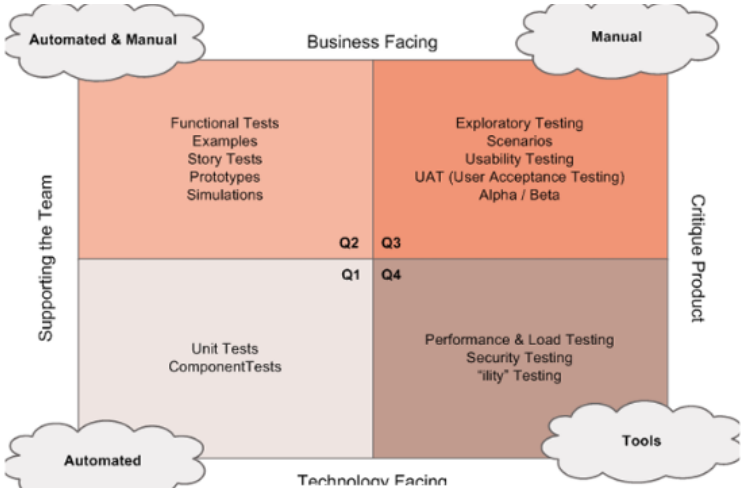
Los cuadrantes permiten clasificar las pruebas segun su proposito y orientacion, organizando los tipos de prueba y su momento de ejecucion.

Q1: comprobar aspectos tecnicos para ayudar al desarrollo

Q2:Comprobar comportamiento esperado para ayudar al desarrollo

Q3: Evaluar producto desde la experiencia del usuario

Q4: Evaluar caracteristicas tecnicas del producto (No funcionales)



EXTENSIONES DE SCRUM

Existen frameworks para escalar la cantidad máxima de integrantes en SCRUM para gestionar productos de mayor tamaño, los cuales son Nexus, scale scrum, less y SAFe.

Framework Nexus

Es un framework que coordina entre 3 y 9 equipos Scrum que trabajan sobre un único Product Backlog para entregar un incremento integrado y terminado de un mismo producto.
Su principal objetivo es gestionar las dependencias de tareas entre equipos y los problemas de integración

Requerimiento: hay que evitar que dos equipos tengan responsabilidades sobre la misma funcionalidad o parte del sistema
el conocimiento del dominio esté distribuido de manera equilibrada entre los integrantes y equipos, para evitar que el trabajo dependa demasiado de una sola persona o de un solo equipo.

RESPONSABILIDADES / ROLES

● **Nexus Integration Team**: Responsable en asegurar que se produzca un incremento integrado al menos una vez en cada sprint, abarcando restricciones técnicas y no técnicas.

● **PO**: Responsable en maximizar el valor del producto y gestionar el backlog (UNO SOLO)

● **Scrum Master**: Responsable en asegurar el marco de trabajo Nexus se entienda y promueva (UNO POR EQUIPO)

● **Uno o más miembros del Nexus Integration Team**: Ayudan a adoptar practicas y herramientas para lograr entregar un Integrated Increment que cumpla el DoD.

EVENTOS

- Nexus sprint
- Refinamiento del backlog entre equipos
- Nexus sprint planning
- Nexus Sprint Review
- Nexus Sprint Retrospective
- Nexus daily

ARTEFACTOS

- Product backlog
- Nexus Sprint Backlog
- Integrated Increment: Incremento del trabajo integrado del sprint de cada equipo

Framework LeSS

LeSS es un marco de trabajo que permite escalar Scrum y se aplica a productos de entre dos a ochos equipos. LeSS propone un solo Product Owner, un Scrum Master que puede servir de uno a tres equipos, gestionando un único Product Backlog. El Sprint es común para todos los equipos. Si se trabaja con más de ocho equipos, se utiliza LeSS Huge.

PRINCIPIOS • Scrum a gran escala; • Pensamiento sistémico; • pensamiento Lean;
• Control empírico de procesos; • Transparencia; • Mejora continua hacia la perfección;
• Centrado en el consumidor;

MIEMBROS

SCM: Enseña Scrum y LeSS, solo cumple el rol de master
PO: Enseña Scrum y LeSS también, guía a todos a descubrir cómo pueden contribuir mejor a crear e producto
Equipos

EVENTOS

Sprint Planning One: Atendida por el PO y los representantes de los equipos, seleccionan ítems tentativos
Sprint Planning Two: Deciden cómo harán los ítems seleccionados, diseño y creación del Sprint Backlog
Daily: 15 min
Review
Restrospective: Particular por equipo
Overall Restrospective: General entre todos
Product Backlog Refinement: Refinamiento
LeSS Sprint: a nivel de producto, no uno diferente para cada equipo. Todos al mismo tiempo

ARTEFACTOS

Incremento potencialmente entregable: Salida del Sprint, con la integración del trabajo de todos los equipos. depende del DoD común entre TODOS los equipos (aunque estos pueden tener un DoD mas detallado internamente, todos comparten una base)
Product Backlog: un solo backlog, igual que en Scrum
Sprint Backlog: Cada equipo tiene su propio sprint backlog

LeSS Huge

Se diferencia en: Tiene área de Product Manager Área de POs Área de Product Backlogs Área de Product Vision Mas de 8 equipos

FILOSOFIA LEAN Y METODO KANBAN

FILOSOFIA LEAN

Lean es una filosofía de gestión orientada a **maximizar el valor entregado al cliente**, **minimizando el desperdicio**, promoviendo la mejora continua de los procesos.
La gestión no se limita a proyectos, puede ser a empresas, producción, etc
Desperdicio: es todo aquello que consume recursos sin aportar valor significativo (por ejemplo, crear una funcionalidad que el cliente nunca usara).

PRINCIPIOS DE LEAN

1) **Busca eliminar desperdicios**, identificando y reduciendo actividades que no aportan valor
2) **Amplificar el aprendizaje** mediante experimentación, iteraciones cortas, feedback temprano
3) **Diferir compromisos**, cuando no se tiene la suficiente información no tomar decisiones irreversibles hasta último momento, no es demorar el proyecto, sino evitar tomar decisiones con incertidumbre
4) **Entregar lo antes posible**, Busca reducir los tiempos en que el cliente recibe valor haciendo entregas frecuentes para obtener retroalimentación, no acelera el proceso, hace entregas parciales
5) **Dar poder al equipo**, Otorgar autonomía y responsabilidad. Las decisiones tienen que ser llevadas a cabo por aquel que HAGA el trabajo
6) **Embeber la integridad conceptual**, Mantener coherencia en el diseño y construir calidad desde el inicio, tener un criterio en común
7) **Ver el todo**, entender el producto que se esta construyendo para entregar valor.
Para optimizar un proceso, primero se debe entender que se busca ya que es riesgoso eliminar desperdicios si no se entiende que se está haciendo

DESPERDICIO LEAN TRADICIONAL

El desperdicio es aquello que no aporta valor significativo, lo cual puede ocurrir por producir en exceso, defectos, tiempos de espera, falta de información, aplicar sobre ingeniería, etc. **Se clasifica en 2 tipos:**
Tipo 1 - Necesario: No aporta valor directo, pero es necesario bajo condiciones actuales. Se busca MINIMIZAR este desperdicio para que tenga el menor impacto posible (ej: managers o niveles de supervisión en los procesos {intermedios}, los que generan burocracia).
Tipo 2 - Puro: Directamente no aporta ningún valor ni es necesario. se busca ELIMINAR

7+1 DESPERDICIO LEAN TRADICIONAL

Desperdicio Lean tradicional	Equivalente en software	Explicación y ejemplo
Talento no utilizado (+1)	Talento no utilizado (+1)	No aprovechar habilidades, experiencia y creatividad del equipo. Ej: ignorar las propuestas de mejora de los testers.
Inventario	Trabajo parcialmente terminado	Trabajo iniciado que todavía no genera valor entregado. Ej: código desarrollado, pero sin integrar ni probar.
Movimiento	Cambio de contexto	Alternar continuamente entre actividades, perdiendo concentración y tiempo al retomarlas. Ej: interrumpir desarrollo para atender incidencias. Pérdida de productividad por tiempo de seteo. (PEOR FUENTE)
Esperas	Esperas y demoras	Trabajo detenido por dependencias o aprobaciones. Ej: historia bloqueada a la espera de validación del cliente. El PO no tiene tiempo para las validaciones; no se aceptan los Pull request; que el cliente apruebe
Transporte ~~~ JIT intenta evitar gastos de stock~~	Cambio de tareas / transferencia de conocimiento	Traspasos entre personas o equipos que requieren coordinación, capacitacion y generan pérdida de contexto. Ej: explicar repetidamente una tarea a distintos equipos. Agregar nuevas personas. Multitasking
Defectos	Defectos	Productos de trabajo con errores que requieren corrección y retrabajo. Ej: código defectuoso que exige modificaciones y repetir pruebas.
Sobreproducción (saturar al mercado)	Características extras	Desarrollar funcionalidades no solicitadas o que no aportan valor. Ej: implementar reportes que nadie necesita.
Sobreprocesamiento	Procesos extra	Ejecutar pasos, controles o procedimientos innecesarios. Ej.: aprobaciones redundantes para un cambio sencillo.

RELACION ENTRE LEAN Y KANBAN

Lean es la filosofía general mientras que Kanban aporta un método para visualizar, gestionar y mejorar el flujo de trabajo, incluyendo límites al trabajo en progreso (WIP).

KANBAN

Kanban es un método permite gestionar y mejorar servicios que entregan trabajo del conocimiento.
Su propósito es gestionar cómo fluye el trabajo a través de un proceso, identificar problemas y mejorar progresivamente la entrega de valor al cliente.
Kan – Ban = tarjeta – Señalizadora, es la forma de identificar un trabajo que hay que hacer dentro del tablero
Se centra en GESTIONAR EL FLUJO DE TRABAJO.

La práctica base es “visualizar el trabajo” y para esto se utiliza un tablero, dentro de este tablero hay tarjetas que representan el trabajo que hay que hacer

VALORES KANBAN

Transparencia: Hacer visible el trabajo, su estado y problemas (mediante el tablero)
Equilibrio: Equilibrar la demanda del trabajo con la capacidad del sistema
Colaboración: Integrantes trabajan juntos para que las tareas avancen y mejorar el flujo
Foco en el cliente: Priorizar la entrega de valor y necesidades del cliente
Flujo: el trabajo tiene que ocurrir a través de un flujo continuo.
En KANBAN no se trabaja con proyecto, a diferencia de Scrum que trabaja con sprints, con tiempo limitado, timebox).
Liderazgo: Promover iniciativas de mejora desde todos los niveles (cualquier integrante puede impulsar mejoras)
Entendimiento: Comprender como funciona el proceso y cuáles son sus problemas
Acuerdo: Establecer compromisos compartidos sobre como trabajar y como introducir mejoras
Respeto: Reconocer y valorar a las personas y sus responsabilidades.

PRINCIPIOS KANBAN

Principios de gestión del cambio

● Comenzar con lo que haces ahora: Analizar el proceso existente actual antes de intentar transformarlo. No obliga a cambiar inmediatamente la forma de trabajar.
Lo que se hace ahora es lo que se mapea en el tablero y se buscan identificar desperdicios, haciendo cambios EVOLUTIVOS
~~● Respetar roles, responsabilidades y puestos actuales: Se parte de la organización existente sin imponer una nueva estructura~~
● Acordar la búsqueda de la mejora continua mediante el cambio evolutivo: Introducir mejoras graduales, evaluando resultados
● Fomentar el liderazgo en todos los niveles: Cualquier integrante puede impulsar mejora, no solamente los responsables jerárquicos.

Principios de entrega de servicios

● Comprender y enfocarse en las necesidades y expectativas del cliente: El trabajo debe orientarse a la entrega de valor
● Gestionar el trabajo y permitir la autoorganización: Se gestiona el flujo y se deja que las personas organicen como realizar el trabajo
● Revisar periódicamente los servicios y sus políticas para mejorar los resultados entregados: Evaluar funcionamiento de sistema para mejorar resultados

La principal diferencia con el anterior es que la gestión del cambio explican como introducir y sostener los cambios, mientas que la entrega de servicios indica como gestionar el servicio que se entrega al cliente.

PRACTICAS GENERALES DE KANBAN (6 PRACTICAS)

Visualizar el trabajo (LA PRACTICA!)

Representar el proceso mediante un tablero, haciendo visibles los estados, tareas y posibles bloqueos.
Esto permite compartir información, facilitar la cooperación y tomar decisiones rápidamente.

- No es el mismo tablero que SCRUM, gestionan de forma distinta (SCRUM es TODO, DOING, DONE).
- NO existen dos tableros iguales, pueden ser similares. Cada PASO del proceso tiene su COLUMNA (cola)

Limitar el trabajo en curso (WIP)

establecer una cantidad máxima de elementos que pueden encontrarse en progreso en una etapa del proceso. Esto permite evitar la acumulación de tareas en las colas, reducir los tiempos de espera y prevenir cuellos de botella. Se busca priorizar el flujo continuo de trabajo en lugar de mantener a todos los recursos ocupados constantemente.
En Scrum para esto se utiliza el TimeBox

Pull no push

Gestionar el flujo

Buscar que el trabajo avance de forma fluida, predecible y sostenible desde que se solicita hasta que se entrega.
Detectar cuellos de botella, administrar colas y medir el avance. La información de esta práctica permite manejar las expectativas de los clientes, hacer predicciones y mejoras.
Administración de colas: Deja de empezar y empieza a terminar, hacer foco en lo que se necesita, analizándolo de derecha a izquierda (porque el WIP debe liberarse de la derecha hacia la izquierda para permitir el avance del trabajo)

Hacer explicitas las políticas

Definir reglas claras y visibles para gestionar el proceso:
Cuando entra una tarea - cuando puede avanzar - como se prioriza y que limites se aplican.
Es decir, las políticas deben ser pocas, sencillas, bien definidas, aplicables y fáciles de modificar por quienes prestan el servicio

Establecer ciclos de retroalimentación

Revisar periódicamente los resultados del trabajo y del proceso mediante reuniones y cadencias, para detectar problemas y tomar decisiones de mejora

Mejorar colaborativamente, evolucionar experimentalmente

Introducir cambios evolutivos mediante experimentos acordados por el equipo. Si una mejora funciona, mantenerla, sino revertirla

Cola de Producto	Análisis		Desarrollo		Listo para Build	En Testing		En Producción
	En progreso	Hecho	En progreso	Hecho		En Progreso	Listo para Despliegue	

Las columnas HECHO sirven para ayudar al sistema PULL → En caso de cuello de botella por WIP, las personas de la “derecha” pueden pasar a realizar pasos de la “izquierda”.
Se debe poner WIP en la cola de producto PARA evitar que se acumulen funcionalidades infinitamente

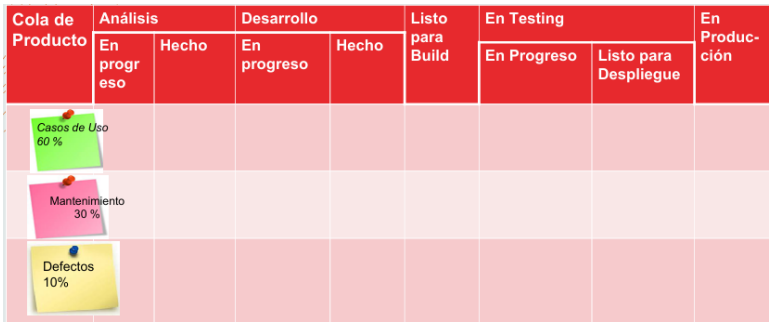
DEFINIR TIPOS DE TRABAJO

- **Requerimiento**: Son funcionalidades nuevas que se quieren incorporar al sistema
- **Defectos**: Son errores del sistema que deben corregirse. (botón que no funciona correctamente en produc)
- **Desarrollo y Mantenimiento**: Incluye tareas como refactorización, mantenimiento y actualización de la infraestructura

Solicitudes: Peticiones de cambio o sugerencias de mejora que pueden surgir durante el uso del sistema

La idea es clasificar el trabajo antes de asignarle capacidad al equipo, porque no todas las tareas tienen el mismo origen ni la misma prioridad.

La capacidad se asigna en funcion de la demanda: Significa decidir qué proporción del tiempo o de los recursos del equipo se destinará a cada tipo de trabajo
Para los casos de uso se destina un 60%
Para los Mantenimiento se destina un 30% (si se tuvieran 100hs de trabajo seria 60 – 30 – 10)
Para los defectos se destina un 10%



POLITICAS EXPLICITAS

Las politicas explicitas son reglas acordadadas que determinan como debe gestionarse el trabajo dentro del sistema kanban, permitiendo que todos conozcan las condiciones de trabajo (inicia, desarrolla, prioriza y finaliza una tarea). Deben ser:
- Pocas, sencillas y bien definidas
- Visibles y acordadas entre las partes involucradas
- Aplicables en todo momento
- Fácilmente modificables por quienes prestar el servicio

Ejemplos son reglas para incorporar tareas al tablero, criterios para avanzar entre estados, los limites WIP, las clases de servicio y las cadencias de reuniones; el DoD; que cada US tenga su prototipo; DoR

APLICACIÓN DE KANBAN

Proceso: Modelar el proceso
Representar las etapas reales por las cuales atraviesa un elemento de trabajo, desde que ingresa hasta que se entrega.
Cola, análisis, desarrollo, listo para build, testing, producción → columnas del tablero

Trabajo: decidir la unidad de trabajo.
Determinar qué representar mediante tarjetas; por ej. CU, US, características, defectos, etc.

Límites de WIP: limitar el WIP para ayudar al flujo de trabajo.
Cantidad maxima de elementos que pueden estar en progreso en cada estado, buscando controlar el trabajo simultaneo para mantener un flujo equilibrado y predecible

Política: definir políticas de calidad.
a) Reposición de trabajo: Cuando ingresan nuevas tareas, cuantas y quien lo decide
b) Criterios de tracción: Cuando una act se considera terminada y puede avanzar al sig estado (no es DoD)
c) Limites WIP: Cuantos elementos pueden estar simultáneamente en un estado
d) Clases de servicio: Reglas que se aplican a diferentes tipos de trabajo
e) Cadencias de reuniones: Horarios, frecuencia y agendas de las reuniones
f) Acuerdos de colaboración: Otras reglas compartidas

Cuellos de Botella y Flujo: mover recursos a los cuellos de botella.

Clase de Servicio: diferentes trabajos tienen diferentes políticas – definición de hecho ("done"), para cada estado.

Cadencia: Releases, planificaciones, revisiones
Reuniones o actividades periódicas que permiten coordinar el trabajo y revisar el funcionamiento del servicio y promover mejoras. Como no hay un evento "final", se debe establecer CUANDO hacer.

Release o entregas: Coordinar entrega al cliente del trabajo terminado
Revisión: Examinar el flujo, detectar problemas, evaluar mejoras.
Planificación: Determinar que trabajo incorporar y como organizarlo según la capacidad y demanda.

Cadencia	Ejemplo de frecuencia	Propósito
Team Kanban Meeting	Diaria	Observar y seguir el estado y flujo del trabajo (no de los trabajadores). ¿Cómo podemos entregar los elementos de trabajo más rápido en el sistema? ¿Hay capacidad disponible? ¿Qué debemos tomar a continuación?
Team Retrospective	Quincenal o mensual	Reflexionar sobre cómo el equipo gestiona su trabajo y cómo pueden mejorar
Team Replenishment Meeting	Semanalmente o a demanda	Seleccionar los elementos de la lista de trabajo para realizar a continuación

METRICAS KANBAN

Las métricas permiten evaluar cómo fluye el trabajo, detectar demoras, estimar tiempos de entrega y mejorar la previsibilidad del proceso.

Lead Time: Vista del CLIENTE
Cycle Time: Vista del EQUIPO

Lead Time - Tiempo de entrega

Tiempo que transcurre desde el momento entra un reclamo hasta que se resuelve.
Incluye los periodos en que el trabajo está en cola, se desarrolla, prueba o permanece bloqueado.
1 año y 1 semana

Cycle Time - Tiempo del ciclo

Es el tiempo que conlleva resolver un reclamo 1 semana
Ayuda a evaluar capacidad y rito de terminación del equipo.
CycleTime = LeadTime – Espera en Backlog

Touch Time - Tiempo de trabajo efectivo

Tiempo durante el cual el equipo **trabaja efectivamente** sobre un elemento, es el tiempo en que el reclamo fue realmente "tocado", excluyendo tiempos de espera o de bloqueo.
Cuanto tiempo paso en columnas de "trabajo en curso"

EFICIENCIA DEL CICLO DE PROCESO

La eficiencia permite identificar que proporción del tiempo corresponde al trabajo efectivo frente al tiempo dedicado a esperas u otras interrupciones

$$Eficiencia = \frac{TouchTime}{LeadTime} \times 100$$

METRICAS ORIENTADAS AL SERVICIO

Métricas que permiten evaluar qué nivel de servicio espera el cliente, qué capacidad tiene le sistema y qué compromisos se establecen

- **Expectativa de nivel de servicio (SLE)**: Nivel de servicio que el cliente espera recibir
- **Capacidad de nivel de servicio (SLC):** Nivel de servicio que el sistema realmente es capaz de entregar
- **Acuerdo de nivel de servicio (SLA):** Nivel de servicio acordado con el cliente
- **Umbral de adecuación de servicio**: Nivel minimo por debajo del cual el servicio esulta inaceptable por el cliente

DIFERENCIAS ENTRE SCRUM Y KANBAN

Entre ambos frameworks, no difieren en filosofía sino en la IMPLEMENTACION, las prácticas de cómo llevar a cabo las variables

Aspecto	Scrum	Kanban
Tiempo	Sprint, TimeBox	Cadencias variables
Compromiso	Compromiso de trabajo por iteración	Opcional
Métrica	Velocidad	Lead Time promedio
Equipos	Multifuncionales	Multifuncionales o no
Seguimiento	Burn-down	No hay
Limite WIP	Indirecto, capacidad	Explicito
Agregar trabajo	Se protege sprint goal	Puede, si hay capacidad
Estimaciones	Para planificar	Opcionales
Roles	PO, SC, Dev	No describe
Priorización	Product Backlog ordenado	Opcional
Tablero	Organizado por Sprint	Persistente, representa flujo continuo
Tamaño del trabajo	Se busca completar las US por sprint	No describe

CALIDAD

Se la puede definir formalmente como el cumplimiento de los requerimientos funcionales y performance explícitamente definidos y de las características implícitas esperadas.
Pero en realidad la calidad es un aspecto totalmente subjetivo ya que va de la mano de las expectativas y necesidades de la persona, la misma determina si el producto satisface SUS necesidades y SUS expectativas y en funcion de eso determina la calidad

GESTION DE CALIDAD

La gestión de calidad para los sistemas software tiene tres intereses fundamentales:

• A nivel de organización, la gestión de calidad se ocupa de establecer un marco de proceso y estándares de organización que conducirán a software de mejor calidad.

• A nivel del proceso, la gestión de calidad implica la aplicación de procesos específicos de calidad y la verificación de que continúen dichos procesos.

• A nivel del proyecto, la gestión de calidad se ocupa de establecer un plan de calidad para un proyecto.

ASEGURAMIENTO DE CALIDAD

Es la definición de procesos y estándares que deben conducir a la obtención de productos de alta calidad ~~y, en el proceso de fabricación, a la introducción de procesos de calidad~~.

El aseguramiento de calidad para SW busca actuar como **medida preventiva** a diferencia del testing que llega tarde (busca defectos).

El equipo QA debe ser independiente del equipo de desarrollo para que pueda tener una perspectiva objetiva del software (y no se vea uno influenciado del otro).

La planeación de calidad es el proceso de desarrollar un plan de calidad para un proyecto.
El plan de calidad debe establecer las cualidades deseadas de software y describir cómo se valorarán. ~~Por lo tanto, define lo que realmente significa software de "alta calidad" para un sistema particular~~.

~~Estructura para un plan de calidad (Humphrey 1989):~~
~~1) Introducción al proyecto~~
~~2) Planes del producto: fechas de entrega críticas y las responsabilidades para el producto~~
~~3) Descripciones de procesos~~
~~4) Metas de calidad: Metas y planes de calidad para el producto, incluyendo una identificación y justificación de los atributos esenciales de calidad del producto~~
~~5) Riesgos y gestión de riesgos~~
Me parece bastante al pedo esto a mi Y lo q sigue tmb es biri biri

Aunque los estándares y procesos son importantes, los administradores de calidad deben enfocarse también a desarrollar una **"cultura de calidad"** donde todo responsable del desarrollo del software se comprometa a lograr un alto nivel de calidad del producto.
Deben exhortar a los equipos a asumir la responsabilidad de la calidad de su trabajo y desarrollar nuevos enfoques para el mejoramiento de la calidad.

PROBLEMAS EN LA CALIDAD

Situaciones que ocurren en proyectos de desarrollo de SW

• **Atrasos en las entregas** -> Relacionado a calidad del proyecto.

• **Requerimientos no claros** -> Relacionado a calidad del producto.

• **Software que no hace lo que debería** -> Relacionado a calidad del producto.

• **Costos excedidos** -> Relacionado a calidad del proyecto.

• **Trabajo fuera de hora** -> Relacionado a calidad del proyecto.

• **Fenómeno 90-90** -> Relacionado a calidad del proyecto.
(90% hecho – 90% faltante → se pregunta como viene el trabajo y se responde que esta casi listo pero nunca se transforma en listo)

• **No se aplica SCM**. -> Relacionado a calidad del producto.

Un software de calidad no solo debería cumplir con los requerimientos, sino que también debería poder entregarse a tiempo, no exceder costos, etc. (Si esto no se cumple genera en el cliente una sensación de falta de calidad)



Situaciones que evidencian NO calidad

La calidad tiene que ver con, el producto, proyecto, la gente, y todo lo que está afectado por hacer SW
La calidad implica satisfacer mucha gente que tiene expectativas/necesidades:
- Expectativas del cliente
- Expectativas del usuario
- Las necesidades de la gerencia
- Las necesidades del equipo de desarrollo y mantenimiento
- otros interesados

PRINCIPIOS DE CALIDAD

• **La calidad NO se inyecta NI se compra, debe estar embebida**: la calidad es algo que se concibe desde el momento cero, desde el relevamiento de requerimientos, en adelante. Es un mito que con testing se puede agregar calidad, el testing solo sirve para determinar que tan bien o mal este hecho el producto

• Es una responsabilidad de todos los involucrados en el proyecto.

• **Las personas son la clave para lograrla**: se hace hincapié en la capacitación, para brindar herramientas y conocimientos a los involucrados ya que hacer software es una actividad humano-intensiva y el éxito de un proyecto es por la gente, por eso es el aspecto mas caro
"*Si cree que la capacitación es cara, pruebe con la ignorancia*"

• Se necesita sponsor a nivel gerencial, pero se puede empezar por uno.

• Se debe liderar con el ejemplo.

• No se puede controlar lo que no se mide, debemos usar métricas.

• Simplicidad, empezar por lo básico.

• Debe planificarse el aseguramiento de calidad.

• El aumento de las pruebas no aumenta la calidad. La calidad se obtiene y se inyecta a lo largo del proyecto, no al final.

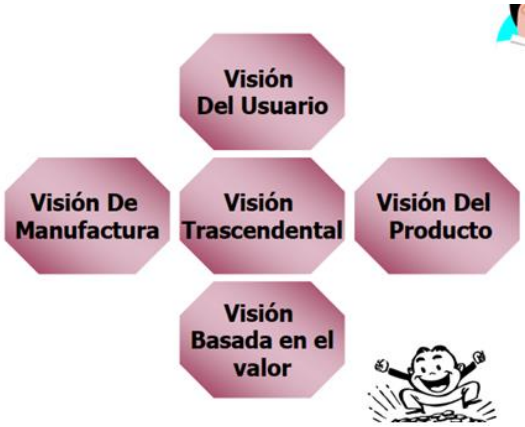
• Debe ser razonable para mi negocio.

• **Prevención mejor que corrección**: prevenir es menos costoso, ya que corregir demanda un costo muy alto asociado al retrabajo.

En la actualidad, el trabajo con calidad es fundamental en la producción del software ya que:
• Es un aspecto competitivo, que permite sobrevivir en el mercado internacional.
• Retiene a los clientes e incrementa los beneficios. Siempre que hablamos de calidad, en el fondo nos referimos al cumplimiento de las expectativas del cliente en todos los sentidos.
• Determina un equilibrio del costo-efectividad. Trabajar con calidad reduce los costos y el retrabajo, ya que se asume que habrá menos fallas.

CALIDAD PARA QUIÉN – VISIONES DE CALIDAD

A la calidad se la puede analizar desde distintas perspectivas:



Visión del usuario: tiene que ver con las expectativas que tiene el usuario del producto, y si este las satisface.
Esta es quizás la más complicada de establecer, porque está en la cabeza de los usuarios, por lo que la comunicación constante es crucial, para ir validando en todo momento si estamos en el camino correcto.

El agilismo trata de incluir la visión de calidad del usuario a través del rol de Product Owner, el cual es ocupado por una persona con capacidad de decisión del cliente.

Visión del producto: esto se asocia al nivel de satisfacción de los requerimientos particulares de cada producto.

Visión del proceso (manufactura): Determina si el proceso de desarrollo utilizado es el correcto para el producto que se desea desarrollar, es decir, el proceso aporta valor al producto y no produce desperdicios

Visión del valor: busca encontrar un equilibrio en la relación costo-beneficio, para obtener siempre el mayor valor para el cliente posible y, generar ganancias con el desarrollo del software.

Visión Trascendental: esta visión es un tanto utópica, ya que se asocia a objetivos difíciles de alcanzar. Por ejemplo 0 defectos en el producto, pero son aquellos objetivos que motivan a seguir en busca de la mejora continua

CALIDAD EN EL SOFTWARE

La gente necesita definir un proceso para llevar a cabo el software, y generalmente en las organizaciones se basan en modelos para tomar de referencia.
Si quiero funcionar con mejora continua tengo modelos para mejorar esos procesos. También tenemos modelos de evaluación que ven el grado de adherencia del proceso al modelo que se tomó de referencia.

Los procesos se instancian en los proyectos. Los procesos son solo una indicación de cómo hacer las cosas, pero es el proyecto que al ejecutarse se insertan actividades para ir regulando si lo que estoy haciendo es lo que se había pensado

FALTA INFO PAG 108 del apunte

CALIDAD DEL PRODUCTO

¿Es el nivel de cumplimiento de requerimientos explicitos????, por lo que si no están o están mal definidos no tenemos contra que validar. Para que los requerimientos nos sirvan deben ser medibles y objetivos

La gestión de calidad en productos son acciones sistemáticas de las organizaciones para lograr calidad, las cuales requieren equilibrio entre:
● Calidad programada: alcances del producto planificados.
● Calidad realizada: lo que realmente se ha desarrollado del producto.
● Calidad necesaria: mínimas características que el producto debe tener, para que este satisfaga los requerimientos.

En el equilibrio obtenemos las expectativas de calidad que esperamos del producto y todo lo que esté por fuera de esta es desperdicio o insatisfacción.



Lo deje en la pag 109

TESTING ¿EN AMBIENTES AGILES?

El testing es una tarea que se realiza en el control de calidad del proceso y del producto, esta tarea permite detectar defectos (esto no garantiza la calidad)
Como la calidad es algo subjetivo que busca satisfacer las necesidades manifiestas (Req funcionales) y expectativas no manifiestas (Req no funcionales) del cliente, surge el testing si bien no garantiza la calidad demuestra los defectos en alguno de estos aspectos

Un desarrollo que implementa aseguramiento de calidad y fue exitoso, conlleva un testing fallido (porque no se deberían encontrar defectos)

El testing se lleva del 15% al 30% del costo del proyecto, si bien es costoso termina ahorrando costos futuros de retrabajo
Es un proceso destructivo porque intenta romper lo que otro construyo con el objetivo de encontrar defectos
Se tiene que ir con una actitud negativa para demostrar que algo es incorrecto

PUD:
- Requerimientos
- Análisis
- Diseño
- Implementación (¿pruebas unitarias?)
- Prueba (Se ocupa de defectos porque se introdujeron en las etapas previas y es necesario codigo para poder realizar las pruebas
- Despliegue

Validación: si es el producto correcto (pensa en MVP validas que sea el prod correcto)
Verificación: si funciona correctamente (lo que construí funciona bien/mal/faltan cosas)